

Concept

NOTE: A lot of the code is commented out, and that is because it was test code or code that eventually went unused. Near the end in the 'out' label, it just jumps to the 'end' label because my logic didn't work for whatever reason in the following loops.

My summing algorithm is as simple as I could get it to be, take the file into the buffer, then go through the buffer and sum every individual number together. Which I do.

It takes the string from some index (tracked by edi and esi registers) and checks if its a newline, then adds them into digits 1, 2, and 3. If digits go unused, it goes to the proper summing category to turn the numbers into their proper form. (ie digit 1 being 5, digit 2 being 1, digit 3 being -1 gets turned into 51).

For printing, there is no value greater than 10000 at the end it divides the values and takes the quotients to print while using the remainders to go onto the next digit, then printing them. It was quite tricky getting the div command to work, as I did not realize at first that edx had to be set.

Code

```
extern printf
section .bss
    descriptor resb 4
    buffer resb 1024
    digit1 resb 1024
    digit2 resb 1024
    digit3 resb 1024
    sum resd 100
    outVar resd 1
    tensVar resd 1
    hundVar resd 1

section .data
    pathname dw "randomInt100.txt", 0
    one db 0, 0

section .text
global main

main:
    mov ebx, pathname
    mov eax, 5
    mov ecx, 0
    int 0x80

    xor eax, eax
```

```
mov [sum], eax
```

```
mov eax, -1  
mov [digit1], eax  
mov [digit2], eax  
mov [digit3], eax  
int 0x80
```

read:

```
mov eax, 3  
mov ebx, 3  
mov ecx, buffer  
mov edx, 1024  
int 0x80
```

```
cmp eax, 0  
je out
```

```
mov esi, buffer  
add esi, eax  
mov edi, buffer
```

digitRead:

```
cmp byte [edi], 10 ;goes to next digit if there is a new line  
je readDigits  
cmp edi, esi ;rereads the buffer if they are equal  
je read
```

```
cmp byte [edi], '0' ;if it is less than zero or arger than nine it skips the digit  
jl next  
cmp byte [edi], '9'  
jg next
```

```
mov eax, [digit1]  
cmp eax, -1  
je addOne
```

```
mov eax, [digit2]  
cmp eax, -1  
je addTwo
```

```
mov eax, [digit3]
cmp eax, -1
je addThree
```

```
;      movzx eax, byte [edi] ;conversion to allow it to not break but for value to get
moved
;      sub eax, '0'
;      add [sum], eax
```

addOne:

```
movzx eax, byte [edi]
mov [digit1], eax
```

```
inc edi
jmp digitRead
```

addTwo:

```
movzx eax, byte[edi]
mov [digit2], eax
```

```
inc edi
jmp digitRead
```

addThree:

```
movzx eax, byte[edi]
mov [digit3], eax
```

```
inc edi
jmp digitRead
```

next:

```
inc edi
jmp digitRead
```

readDigits:

```
mov eax, [digit3]
cmp eax, -1
je twoDigits
```

```
mov eax, [digit3]
sub eax, '0'
add [sum], eax
```

```
mov eax, [digit2]
sub eax, '0'
mov ebx, 10
mul ebx
add [sum], eax
```

```
mov eax, [digit1]
sub eax, '0'
mov ebx, 100
mul ebx
add [sum], eax
```

```
mov eax, -1
mov [digit1], eax
mov [digit2], eax
mov [digit3], eax
```

```
jmp next
```

twoDigits:

```
mov eax, [digit2]
cmp eax, -1
je oneDigit
```

```
mov eax, [digit2]
sub eax, '0'
add [sum], eax
```

```
mov eax, [digit1]
sub eax, '0'
mov ebx, 10
mul ebx
add [sum], eax
```

```
mov eax, -1
mov [digit2], eax
mov [digit1], eax
```

```
jmp next
```

oneDigit:

```
mov eax, [digit1]
```

```
sub eax, '0'
add [sum], eax

mov eax, -1
mov [digit1], eax

jmp next
```

out:

```
    jmp notJunk
;    mov eax, [sum]
;    add eax, '0'
;    mov [sum], eax

;    mov eax, 4
;    mov ebx, 1
;    mov ecx, sum
;    mov edx, 1024
;    int 0x80

;    jmp end

;    mov eax, 200
;    mov ebx, 0
;    mov ecx, 50
;    mov edx, 0
;    idiv ecx
;    add eax, '0'

;    mov [one], eax

;    mov eax, [sum]
;    add eax, '0'
;    mov [outVar], eax

;    mov eax, 4
;    mov ebx, 1
;    mov ecx, outVar
;    mov edx, 1024
;    int 0x80
```

; jmp end

mov eax, 0

mov ebx, [sum]

cmp ebx, 99

jg maxSize

cmp ebx, 9

jg medSize

jmp smallSize

; jmp end

maxSize:

add eax, 100

mov ebx, sum

maxLoop:

sub ebx, eax

cmp ebx, eax

jg maxSize

mov edx, 0

mov ecx, 100

idiv ecx

add eax, '0'

mov [hundVar], eax

; mov eax, 4

; mov ebx, 1

; mov ecx, hundVar

; mov edx, 1024

; int 0x80

jmp end

mov eax, 0

medSize:

add eax, 10

```

        mov ebx, sum

medLoop:
        sub ebx, eax
        cmp ebx, eax
        jg medSize

        mov edx, 0
        mov ecx, 10
        idiv ecx

;        add eax, '0'
;        mov [tensVar], eax
;        mov eax, 4
;        mov ebx, 1
;        mov ecx, tensVar
;        mov edx, 1024
;        int 0x80

        mov eax, 0

smallSize:
        add eax, 1
        mov ebx, sum

smallLoop:
        sub ebx, eax
        cmp ebx, eax
        jg smallSize

;        mov eax, [sum]
;        add eax, '0'

notJunk:
        mov eax, [sum]
        mov edx, 0
        mov ecx, 10000
        div ecx

        add eax, '0'
        mov [outVar], eax

```

```
mov eax, 4
mov ebx, 1
mov ecx, outVar
mov edx, 1024
int 0x80
```

```
    mov eax, [sum]
    mov edx, 0
    mov ecx, 10000
    div ecx
```

```
mov eax, edx
mov edx, 0
mov ecx, 1000
div ecx
```

```
add eax, '0'
mov [outVar], eax
```

```
mov eax, 4
mov ebx, 1
mov ecx, outVar
mov edx, 1024
int 0x80
```

```
    mov eax, [sum]
    mov edx, 0
    mov ecx, 1000
    div ecx
```

```
mov eax, edx
    mov edx, 0
mov ecx, 100
div ecx
```

```
add eax, '0'
mov [outVar], eax
```

```
mov eax, 4
mov ebx, 1
mov ecx, outVar
mov edx, 1024
int 0x80
```

```
mov edx, 0
mov eax, [sum]
mov ecx, 100
div ecx
```

```
mov eax, edx
    mov ecx, 10
    mov edx, 0
    div ecx
    add eax, '0'
    mov [outVar], eax
```

```
mov eax, 4
mov ebx, 1
mov ecx, outVar
mov edx, 1024
int 0x80
```

```
mov edx, 0
mov eax, [sum]
mov ecx, 100
div ecx
```

```
mov eax, edx
mov ecx, 10
mov edx, 0
div ecx
```

```
;
```

```
mov eax, edx
add eax, '0'
mov [outVar], eax
mov eax, 4
mov ebx, 1
mov ecx, outVar
mov edx, 1024
int 0x80
```

```
; mov eax, 4
; mov ebx, 1
; mov ecx, sum
; mov edx, 1024
```

```
;      int 0x80
```

end:

```
;      mov edi, [sum]
;      xor eax, eax
;      call printf
;      mov eax, 0
```

```
      mov eax, 6
      int 0x80
```

```
      mov eax, 1
      mov ebx, 0
      int 0x80
```

Output

Custom file "single.txt"

```
[cward6@linux6 backup]$ nasm -felf64 test.asm && g++ test.o && ./a.out
10738[cward6@linux6 backup]$
```

Provided file "randomInt100.txt"

```
[cward6@linux6 backup]$ nasm -felf64 test.asm && g++ test.o && ./a.out
04679[cward6@linux6 backup]$
```